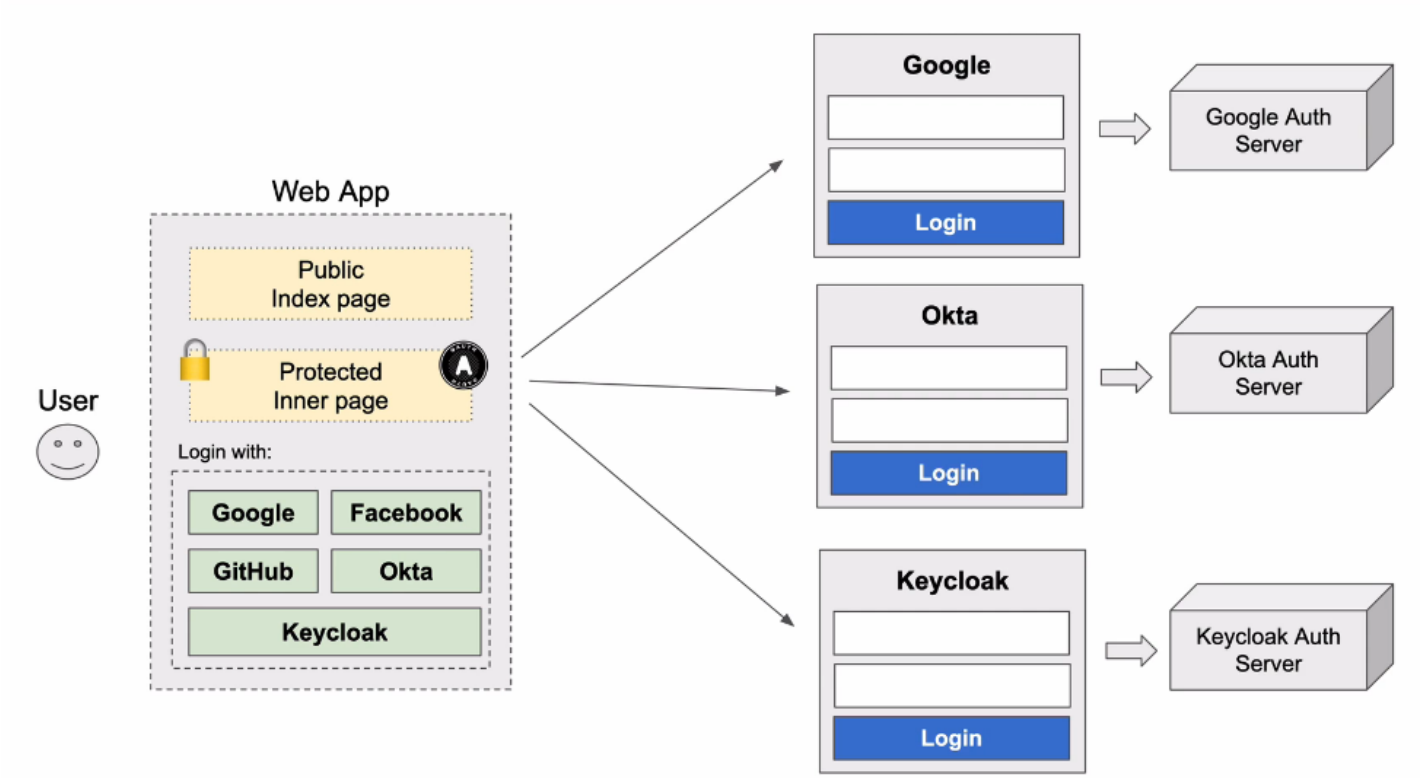
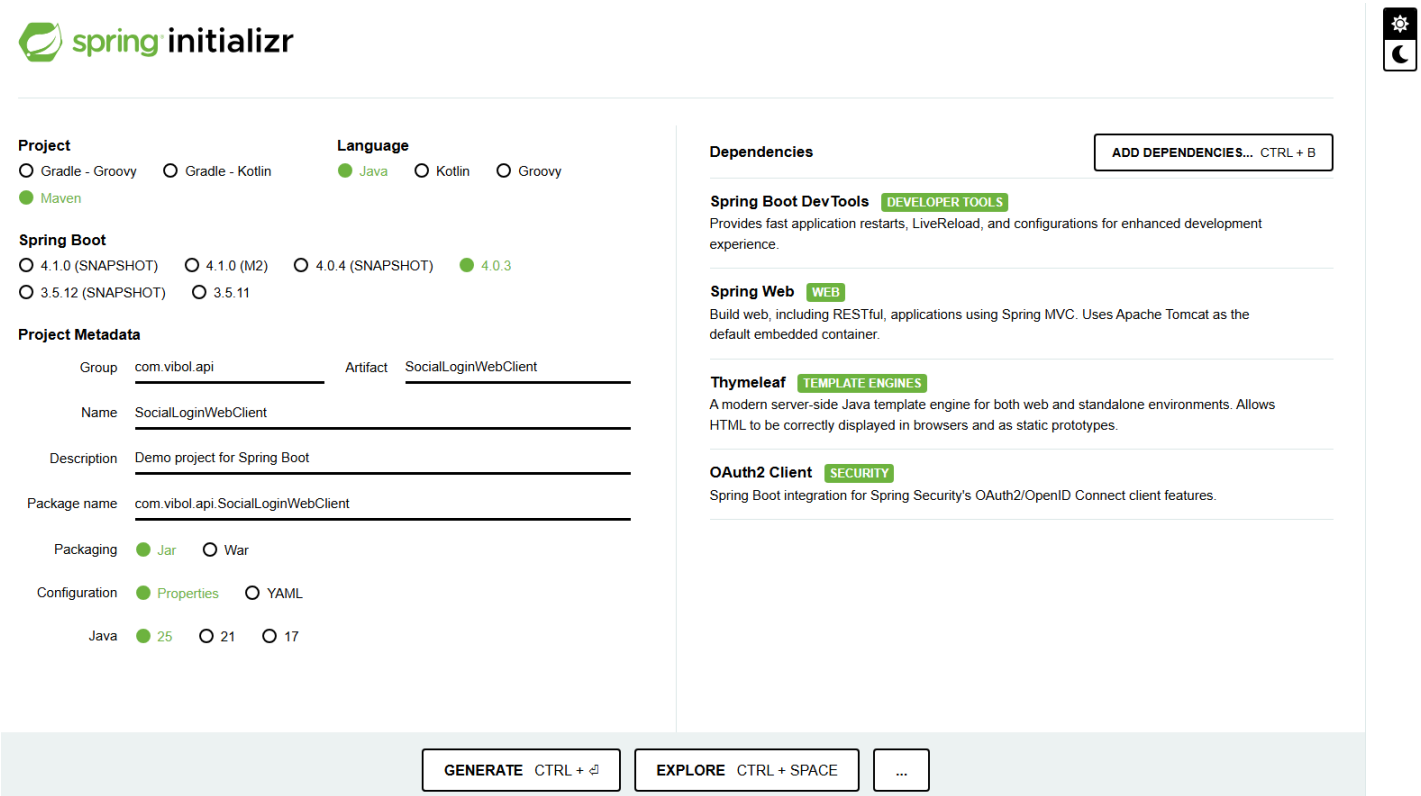
Create Social Login with Spring MVC

យើងមាន Resource Server ដែលអាចអោយ Client ធ្វើការចូលបាន ដោយ http request: one is public and private the user need to login with Authentication Server to Access.



1- Create SocialLoginApp



2- Create Controller:

```java
@Controller
public class HomePageController {

    @GetMapping("/home")
    public String displayHomePage(Model model, @AuthenticationPrincipal OAuth2User principle) {
        if (principle != null) {
            String name = principle.getAttribute("name");
            model.addAttribute("name", name);
        }

        return "name";
    }

}
```

- First injection: @AuthenticationPrincipal OAuth2User principle

ដើម្បីធ្វើការដាក់ Security when user request to this API

- Second : Model model : បង្កើតបោះទៅអោយ Template viewer

តាមរយៈ name ដែលបានមកពី JWT យើងយកវាទៅបង្ហាញនៅក្នុង Template

model.addAttribute("name", name); យើងបោះ parameter នឹង name ទៅអោយ Template.

// html template that we will create: home.html

return "name";

```html
<!DOCTYPE html SYSTEM "http://www.thymeleaf.org/dtd/xhtml1-strict-thymeleaf-4.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:th="http://www.thymeleaf.org">

<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Home page</h1>

    <div>
        Welcome <span th:text="${name}" />
    </div>

</body>
</html>
```

```
<!DOCTYPE html SYSTEM "http://www.thymeleaf.org/dtd/xhtml1-strict-thymeleaf-4.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
        xmlns:th="http://www.thymeleaf.org">

<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
        <h1>Home page</h1>

        <div>
                Welcome <span th:text="${name}" />
        </div>

</body>
</html>
```

2-1 public url

បង្កើត controller for public access then they can click got to home page.

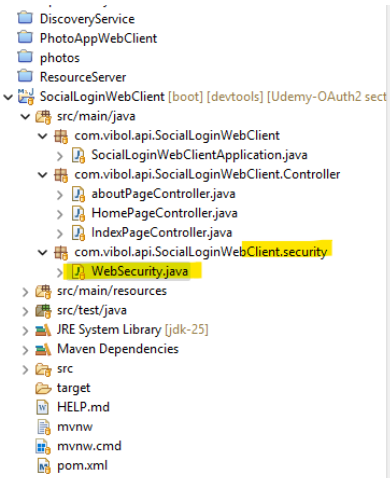
```java
package com.vibol.api.SocialLoginWebClient.Controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class IndexPageController {

    @GetMapping("/")
    public String displayIndexPage(Model model) {

        return "index";

    }

}
```

Template: index.html

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

<div><a href="/home">Home Page</a></div>

</body>
</html>
```

3- Create Security Filter class from WebSecurityConfiguratinAdapter

ចំនុចសំខាន់ដើម្បីធ្វើការដាក់ Security ដោយសារតែ ក្នុង Spring Security យើងប្រើ Webflux ចឹងយើងត្រូវ write code for webflux:




```java
import org.springframework.context.annotation.Bean;

@Configuration
public class WebSecurity {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/js/**", "/force-login").permitAll()
                .anyRequest().authenticated()
        )
        .oauth2Login(oauth -> oauth
                .loginPage("/oauth2/authorization/google")
        )
        .logout(logout -> logout
                .logoutUrl("/logout")
//                .logoutSuccessUrl("/about")
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
        );

        return http.build();
    }
}
```

```
@Configuration
public class WebSecurity {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/js/**", "/force-login").permitAll()
                .anyRequest().authenticated()
        )
        .oauth2Login(oauth -> oauth
                .loginPage("/oauth2/authorization/google")
        )
        .logout(logout -> logout
                .logoutUrl("/logout")
//              .logoutSuccessUrl("/about")
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
        );

        return http.build();
    }
}
```

Explain :

1- .requestMatchers("/", "/css/**", "/js/**", "/force-login").permitAll() អោយតែ api ណា ក៏ដោយមិនចាំបាច់ត្រូវធ្វើការ Authentication ទេ។ Ex: / (root url)

2- .oauth2Login(oauth -> oauth
   .loginPage("/oauth2/authorization/google")
)

   ត្រូវ Request to google login authentication.

```
        .logout(logout -> logout
                .logoutUrl("/logout")
//                .logoutSuccessUrl("/about")
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
        );

        return http.build();
    }
```

ពេល Logout វ៉ាត្រូវធ្វើការ Clear Session នឹងហើយអោយវ៉ាទៅ / (root) វិញ

4- application.propertis

```
spring.application.name=SocialLoginWebClient
server.port=8088

spring.security.oauth2.client.registration.google.client-id=${GOOGLE_CLIENT_ID}
spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_CLIENT_SECRET}

# Facebook
spring.security.oauth2.client.registration.facebook.client-id=${FACEBOOK_CLIENT_ID}
spring.security.oauth2.client.registration.facebook.client-secret=${FACEBOOK_CLIENT_SECRET}

# GitHub
spring.security.oauth2.client.registration.github.client-id=${GITHUB_CLIENT_ID}
spring.security.oauth2.client.registration.github.client-secret=${GITHUB_CLIENT_SECRET}
```

spring.application.name=SocialLoginWebClient

server.port=8088

spring.security.oauth2.client.registration.google.client-id=${GOOGLE_CLIENT_ID}

spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_CLIENT_SECRET}

Facebook

spring.security.oauth2.client.registration.facebook.client-id=${FACEBOOK_CLIENT_ID}

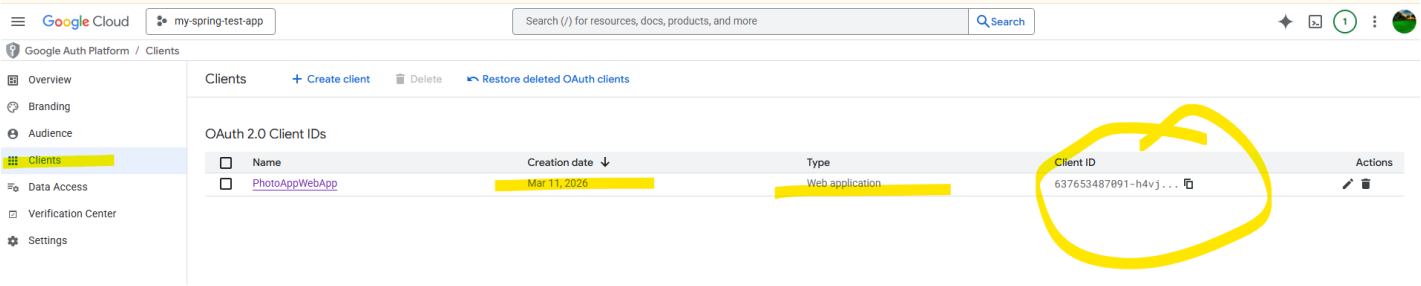
spring.security.oauth2.client.registration.facebook.client-secret=${FACEBOOK_CLIENT_SECRET}

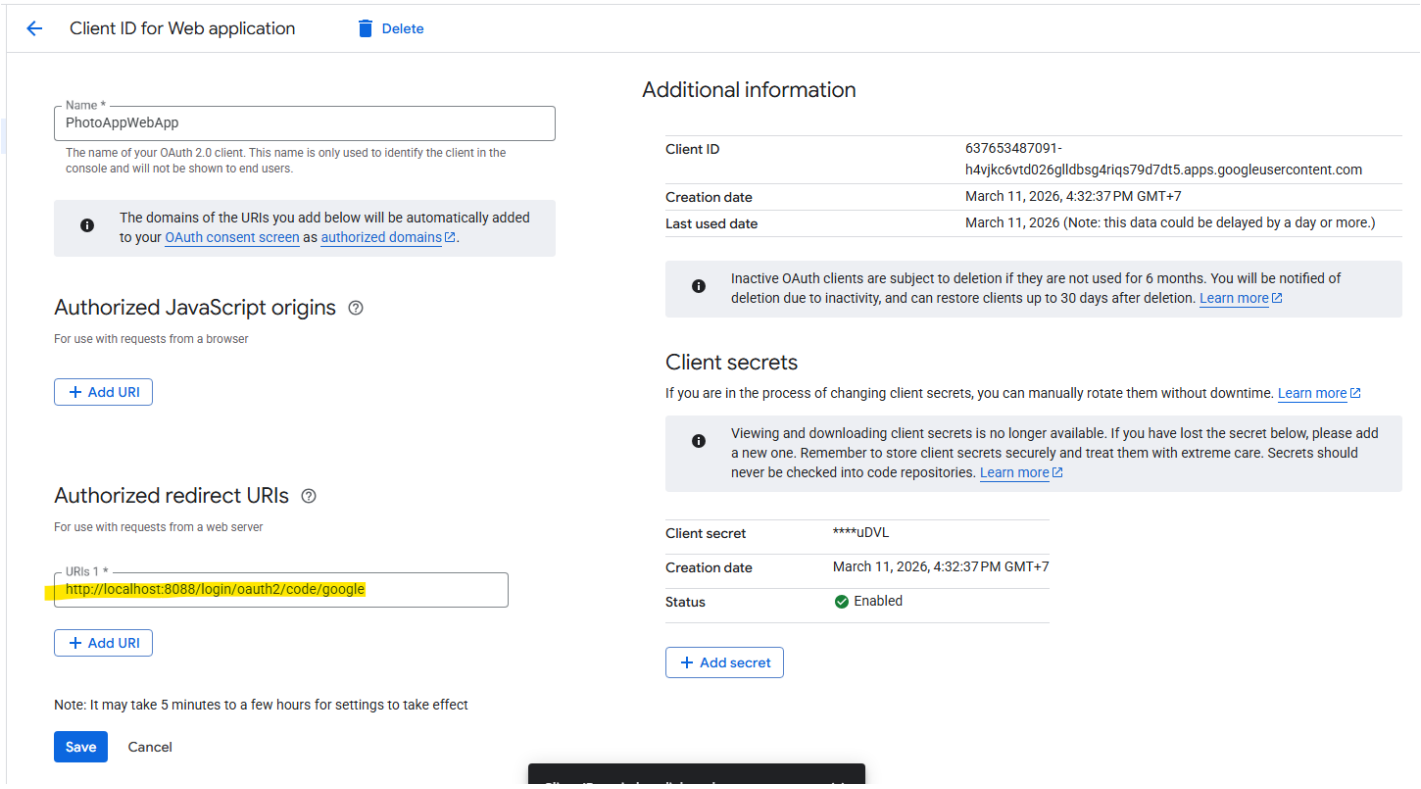
GitHub

spring.security.oauth2.client.registration.github.client-id=${GITHUB_CLIENT_ID}

spring.security.oauth2.client.registration.github.client-secret=${GITHUB_CLIENT_SECRET}

ដើម្បីធ្វើការភ្ជាប់ទៅកាន់ Google Authentication យើងត្រូវបង្កើត google developer account





Authorized redirect URIs: http://localhost:8088/login/oauth2/code/google ជាទីតាំង Url របស់យើង http://localhost:8088 (where the location we running)

Testing Flow

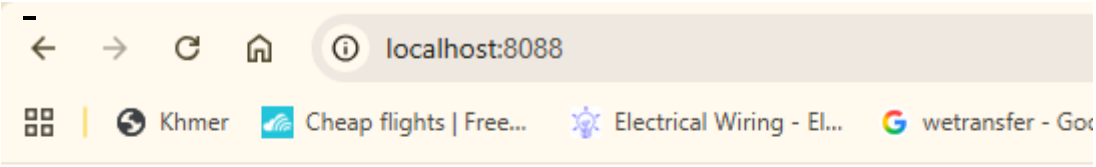
1- User go to http://localhost:8088/

- Controller to the root :

```java
package com.vibol.api.SocialLoginWebClient.Controller;

import org.springframework.stereotype.Controller;

@Controller
public class IndexPageController {

    @GetMapping("/")
    public String displayIndexPage(Model model) {

        return "index";

    }

}

```

- Web Response

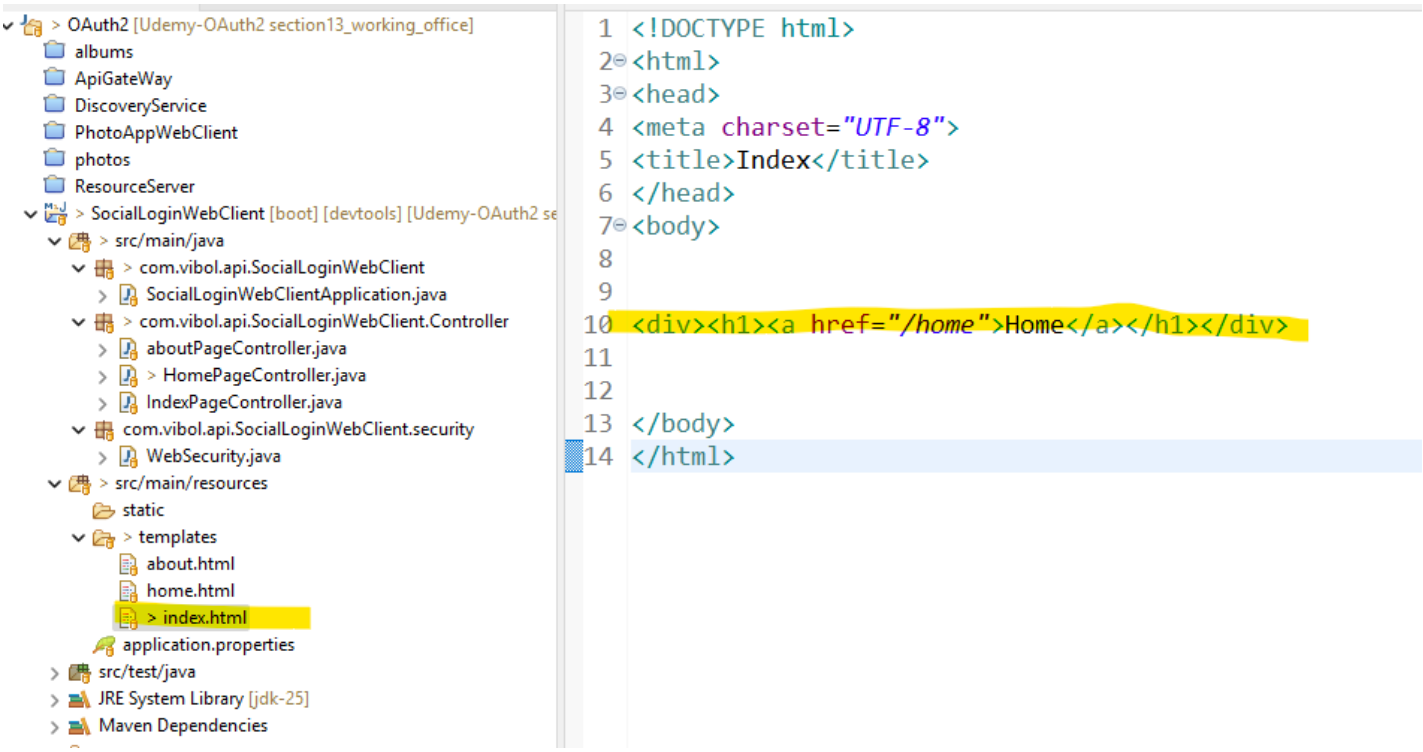
-



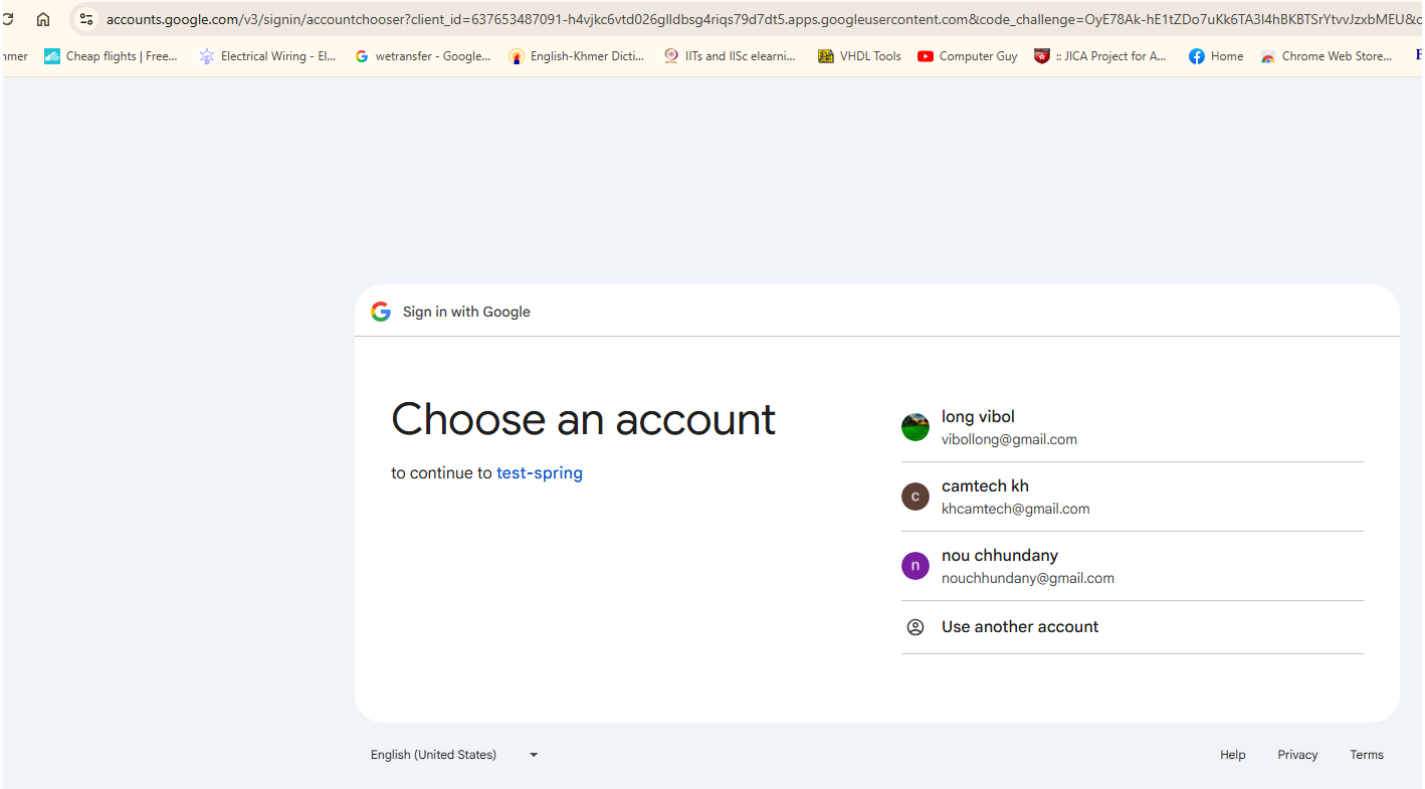
Home

```java
package com.vibol.api.SocialLoginWebClient.Controller;

import org.springframework.stereotype.Controller;

@Controller
public class IndexPageController {

    @GetMapping("/")
    public String displayIndexPage(Model model) {

        return "index";

    }

}
```

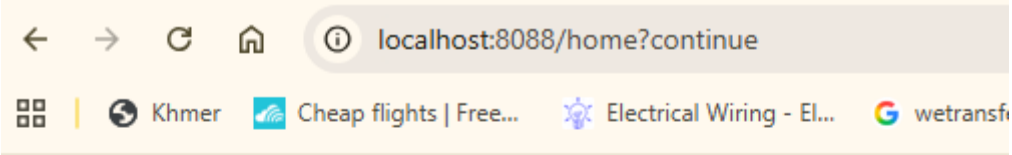
- Html



2- When clink on home: it will directly goto google login page to verify user



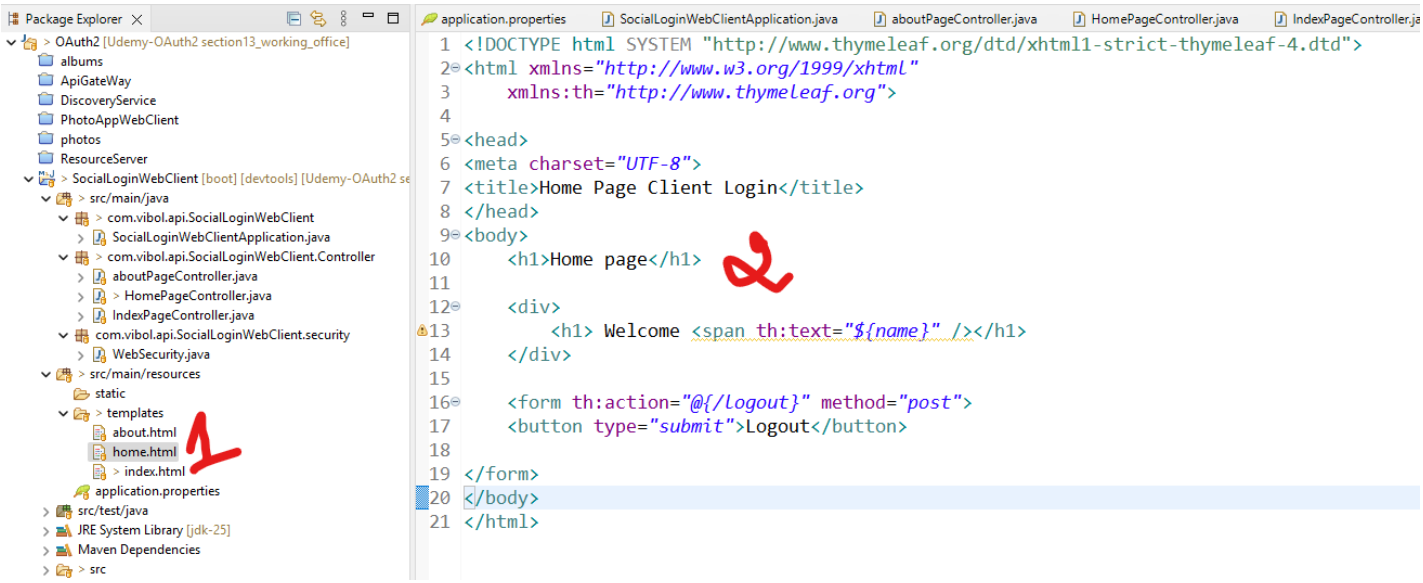
After successfully: it go to the home.html page



Home page

Welcome long vibol

Logout



This is what we protect in the controller /home url

```
package com.vibol.api.SocialLoginWebClient.Controller;

import org.springframework.security.core.annotation.AuthenticationPrincipal;

@Controller
public class HomePageController {

    @GetMapping("/home")
    public String displayHomePage(Model model, @AuthenticationPrincipal OAuth2User principle) {
        if (principle != null) {
            String name = principle.getAttribute("name");
            model.addAttribute("name", name);
        }

        // html template that we will create
        return "home";
    }

}
```

Spring វាប្រាប់ថាទាល់តែមាន@AuthenticationPrincipal OAuth2User principle ទើបអាចចូលបាន

When logout it will go back to root

Home page

Welcome long vibol

Logout

```
import org.springframework.context.annotation.Bean;

@Configuration
public class WebSecurity {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/js/**", "/force-login").permitAll()
                .anyRequest().authenticated()
        )
        .oauth2Login(oauth -> oauth
                .loginPage("/oauth2/authorization/google")
        )
        .logout(logout -> logout
                .logoutUrl("/logout")
//                .logoutSuccessUrl("/about")
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
        );

        return http.build();
    }
}
```